

Lab – Analyzing the WannaCry KillSwitch Using Ghidra

Overview

In this lab, we will reverse engineer the WannaCry Ransomware to examine the killswitch discovered by Marcus Hutchins, aka MalwareTech. Marcus reverse Engineered WannaCry and found the program checks a particular URL that was not registered and inactive. If the domain remained inactive, the ransomware would install. Once Marcus registered the domain, it shut down the ransomware.

May 2017, a ransomware worm quickly spread across the Internet, targeting computers running the Microsoft Windows Operating System.

Once WannaCry gets installed on your Windows machine, it encrypts the files on the PC's hard drive, making them nearly impossible for users to access. WannaCry then demands a ransom to be paid using bitcoins.

Reverse engineering is a crucial process for malware threat intelligence researchers working backward from malware being used to carry out attacks — to understand how the malware works, its capabilities, who wrote it, and where it came from.

Reverse engineering is also essential for defenders to check their code for weaknesses and confirm it works as intended.

Lab Requirements

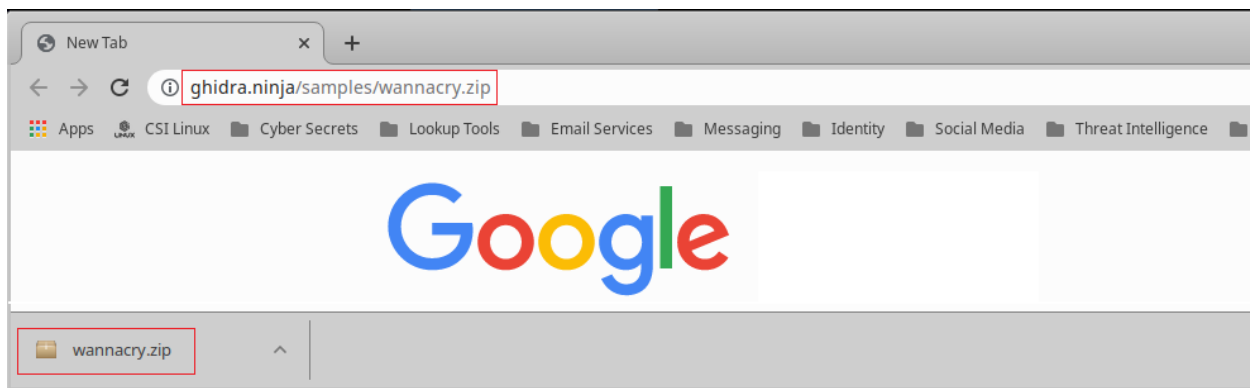
- One installation of VirtualBox with the extension pack.
- One virtual install of the latest version of CSI Linux.
- VirtualBox network adapter set to NAT network.

Download a sample of WannaCry

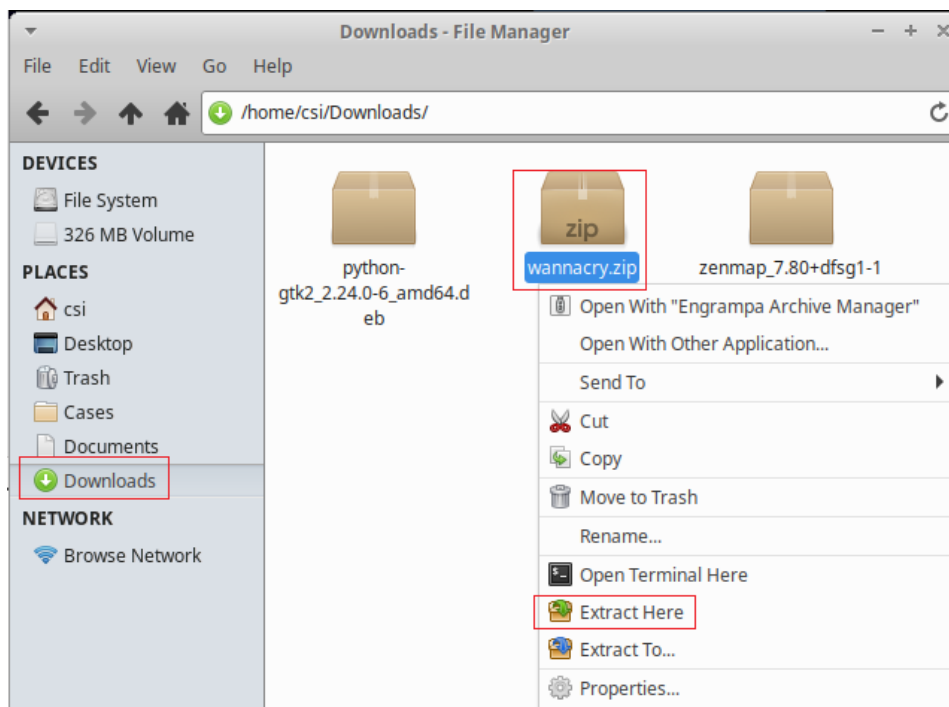
We next need to download an example of WannaCry to analyze.

From your CSI Linux, launch a browser and in the address bar, copy and paste the following URL.

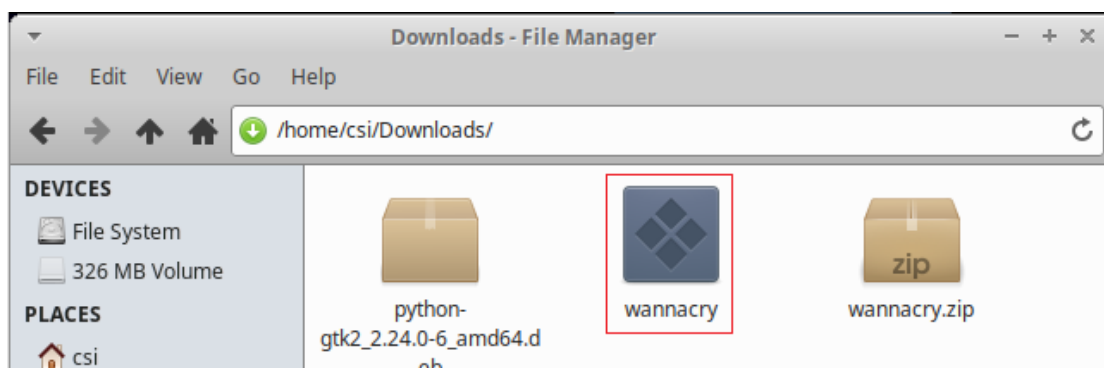
<https://www.ghidra.ninja/samples/wannacry.zip>



Once the download is complete, close your browser. From your desktop, open your Home folder, and from the left-hand windowpane, open your Downloads directory. Find the wannacry.zip archive. Right-click on the archive, and from the context menu select, Extract Here.



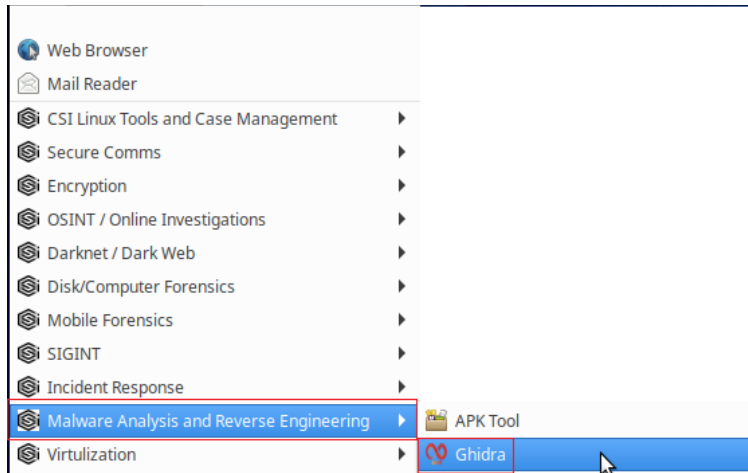
The archive will need the following password. The ZIP's password is **ghidra.ninja**



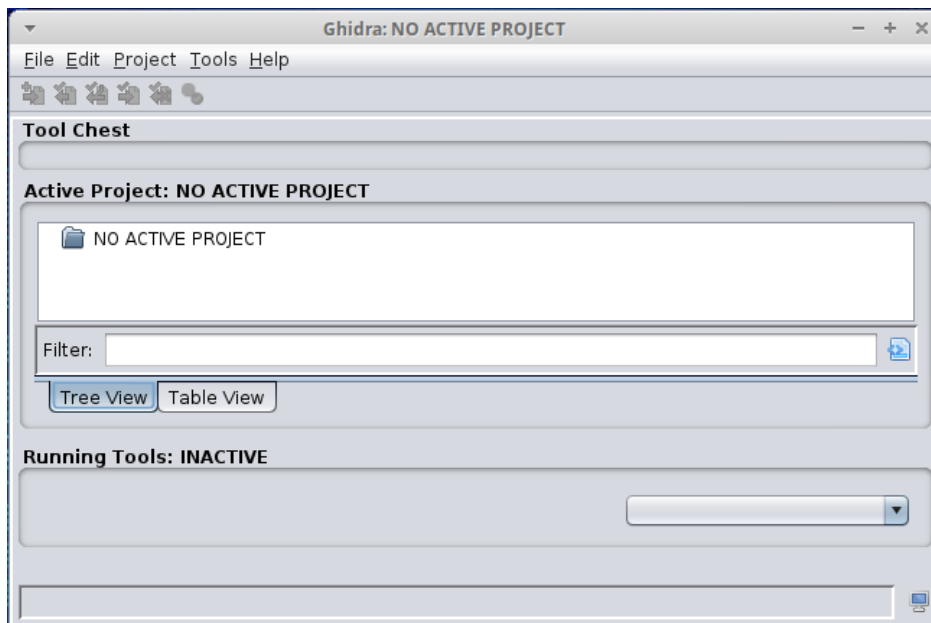
Launch Ghidra

Installing Ghidra onto CSI Linux was covered in a previous [lab](#). This lab assumes you have Ghidra installed.

From the application menu in CSI Linux, scroll down until you come to Malware Analysis and Reverse Engineering. Then, expand the context menu, and from the option, click Ghidra to start the application.

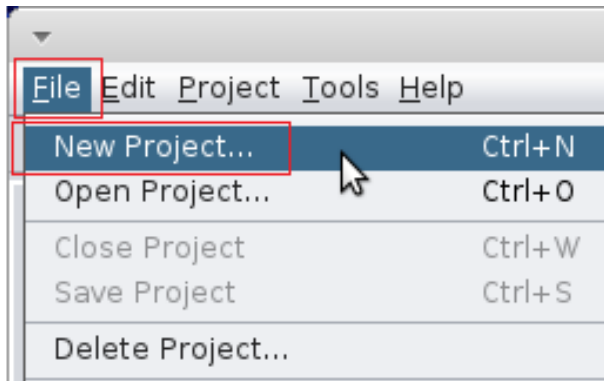


After a short pause, the Active Project screen opens.

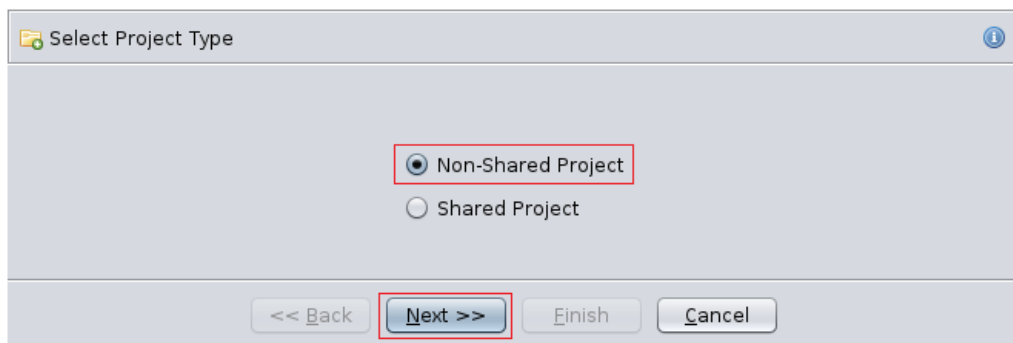


Creating a new project

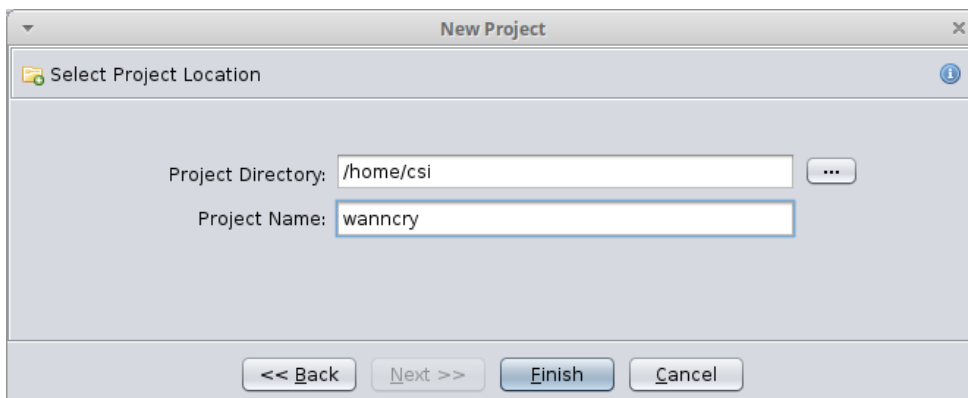
We next need to create a new project. From the Ghidra taskbar, go to File>New Project.



The **Select Project Type** screen opens. Ghidra can be used as a collaboration tool or as a stand-alone, non-shared project. We will be working individually on a project, so click "Non-Shared Project." Then Click Next.

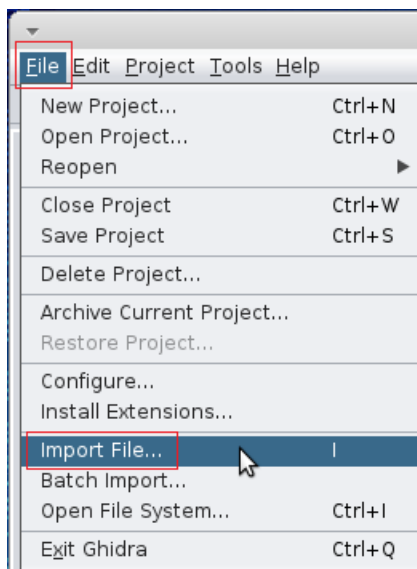


We give our new project a user-friendly name and select a location to save our work on the next screen. In this example, I have named project **wannacry**. I will accept the default location for saving my work. You are free to name your project as you see fit and keep it where you want. Click Finish.

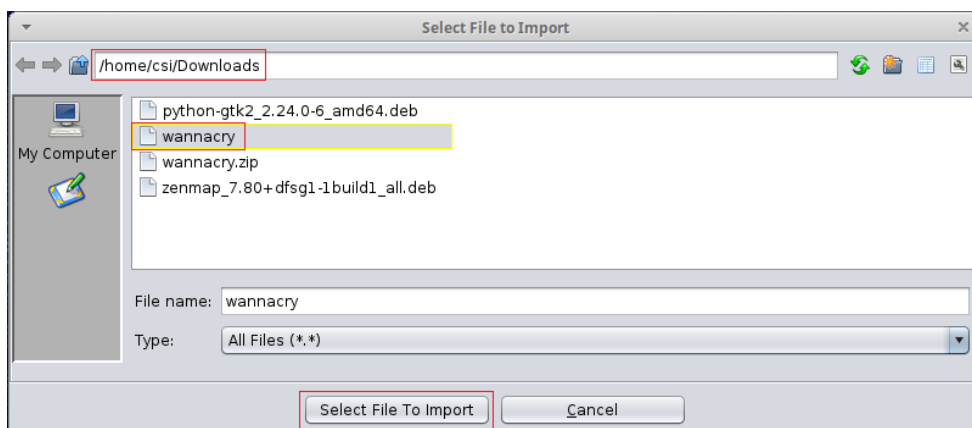


Import wannacry into Ghidra

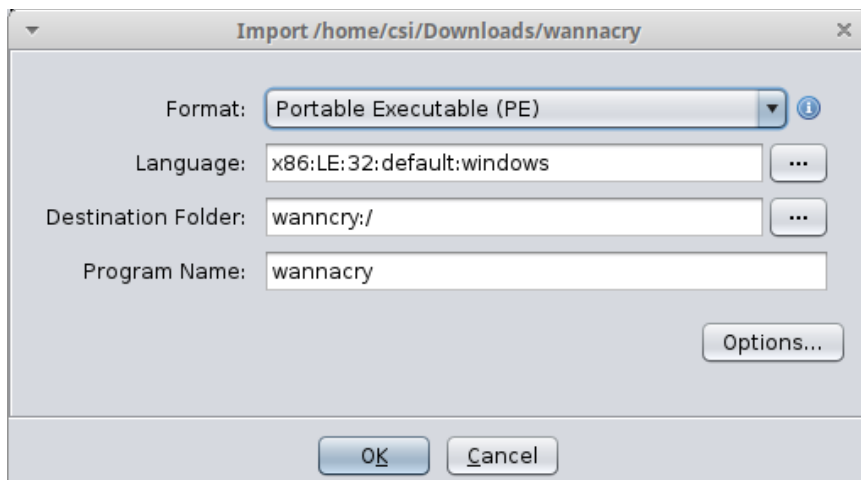
From Ghidra's Active project screen, click on File> Import File.



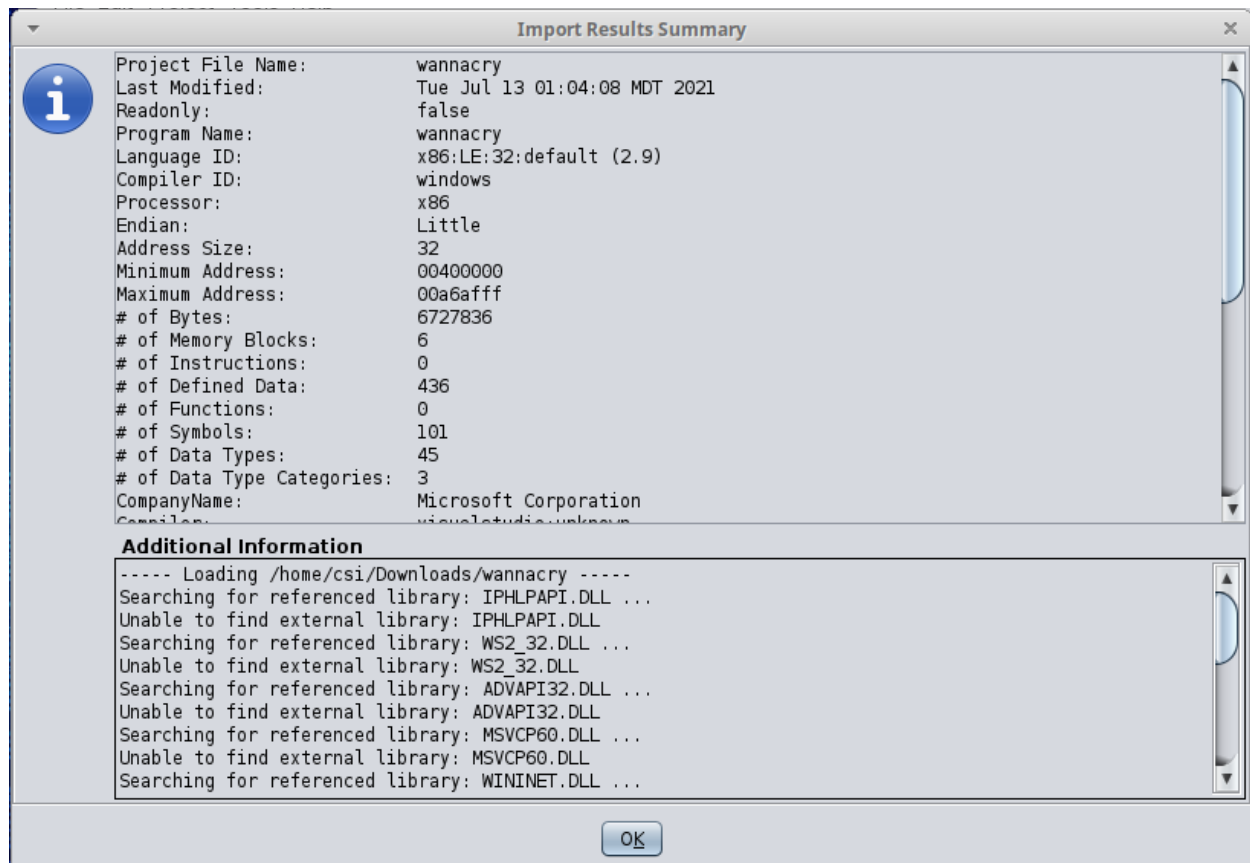
Browse over to your Downloads directory. Find the extracted wannacry file. Highlight the file and press the Select File to Import button.



Once the importation process begins, you are presented with information about the file. Press OK.

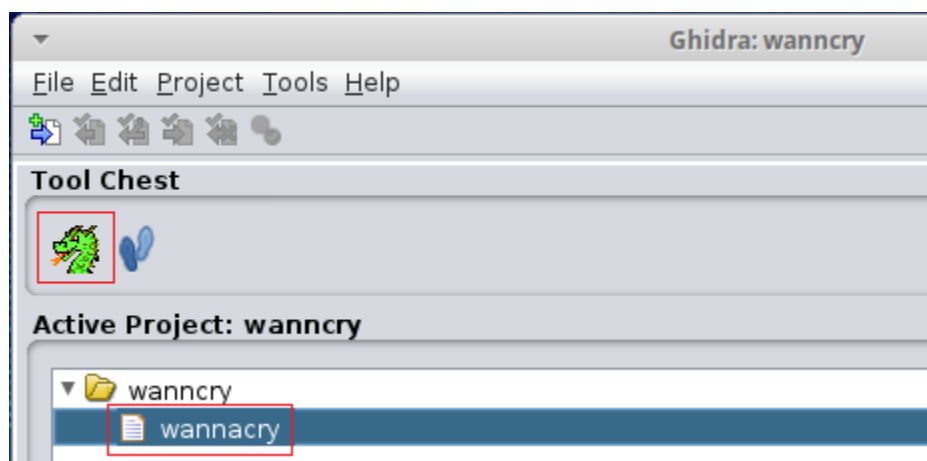


Once the importation process completes, you are given the import summary. Press OK.



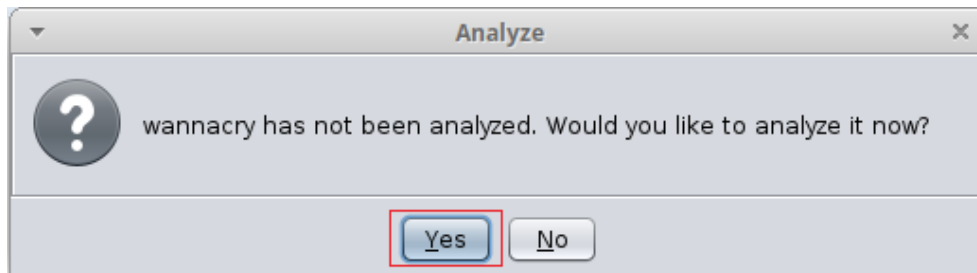
Begin the Analysis

In the Active Project window, highlight the wannacry file and click the green dragon icon to begin the analysis of the file.

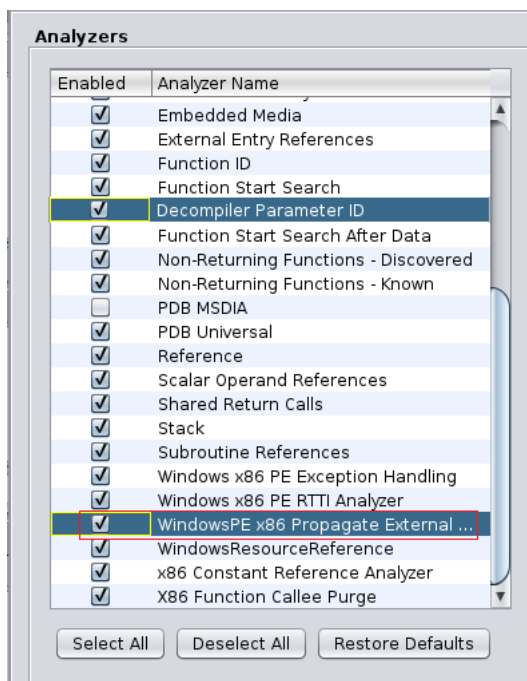


If the code browser shows no content, close the browser, and start the analysis one more time.

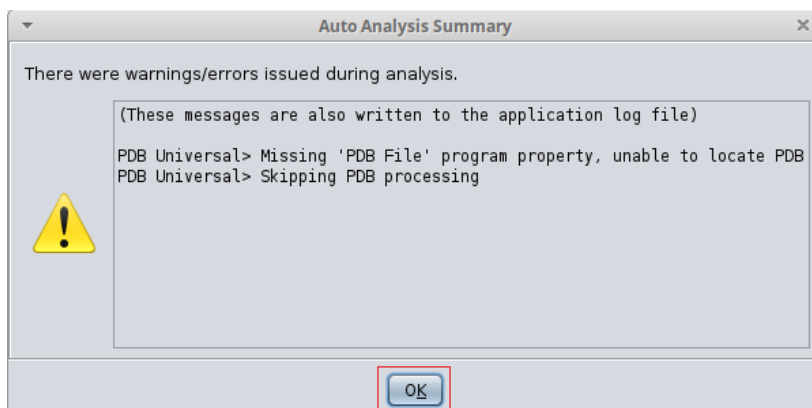
Once the analysis begins, you are asked if you would like to analyze now. Click the Yes button.



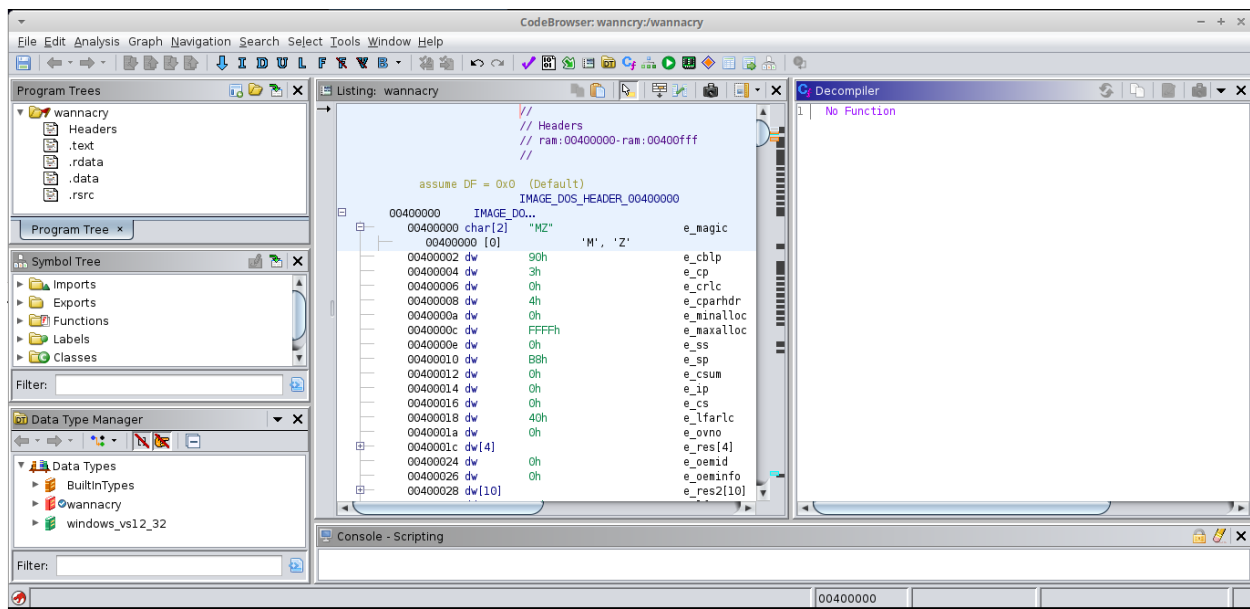
On the next screen, you can select what analyzers to use. Add the Decompiler Parameter ID and WindowsPE x86 Propagate External Parameter option as additional analyzers. Press Analyze.



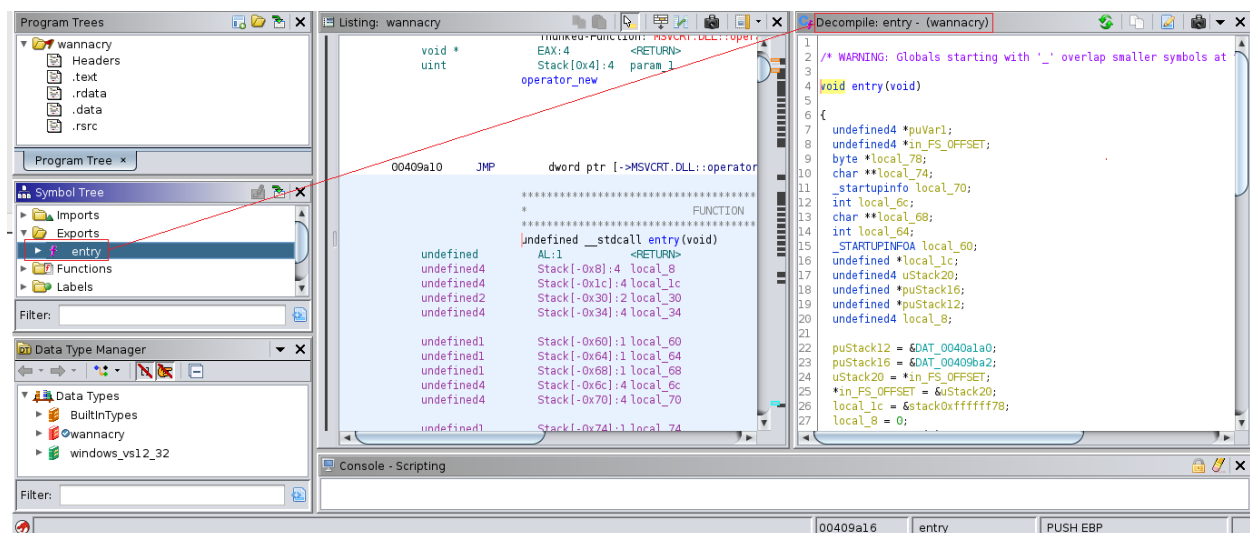
Ignore any warning.



Once the analysis has been completed, you are presented with the compiled code broken out into the four different windows.



We need to find either a `main()` function or a `WinMain()` function. We see neither, but inside the Symbol Tree window, inside the Exports folder, we find a function called **entry**. When we click the **entry** function, it is decompiled, and the code for the function is shown to us in the Decompile window.



What is Winmain?

`WinMain()` is the C entry point function of any windows application. DOS/console-based applications use `main()` as the C programming entry point function; however, windows Win32 graphical applications use `WinMain()` as the entry point function.

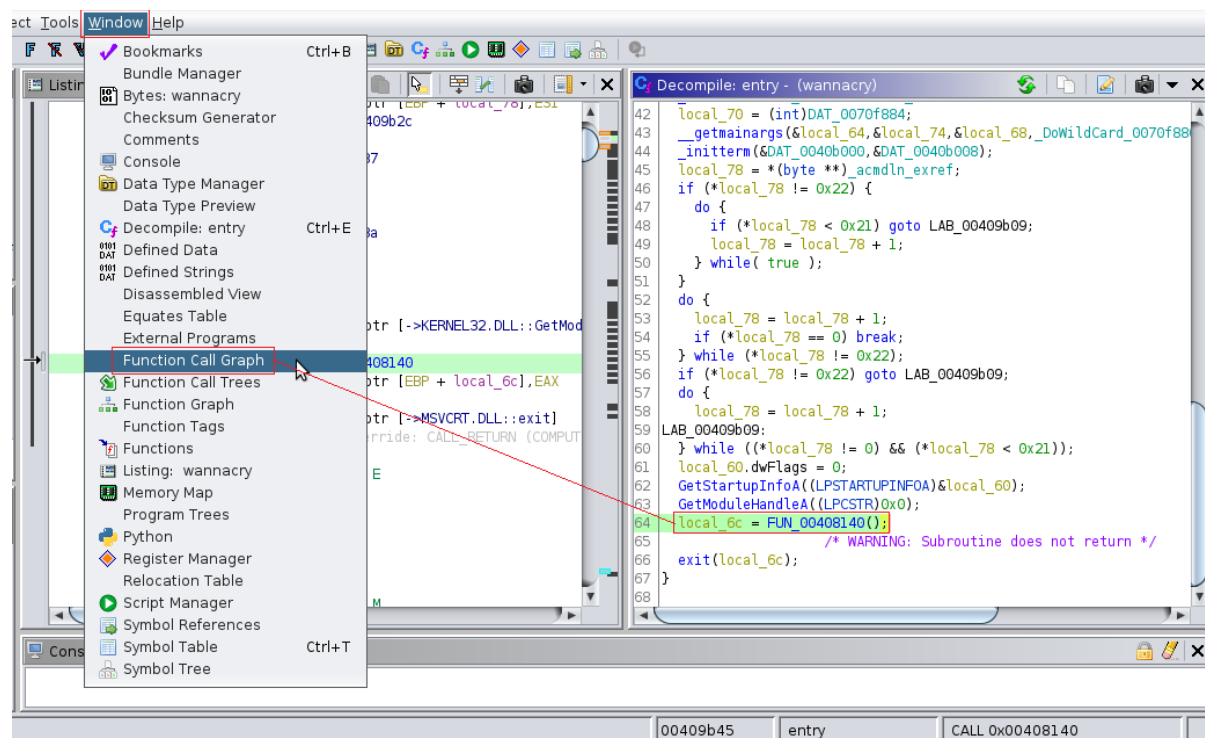
In the Decompiler windows, we scroll to the bottom of the code, where we find a call to another function.

```

59 LAB_00409b09:
60 } while ((*local_78 != 0) && (*local_78 < 0x21));
61 local_60.dwFlags = 0;
62 GetStartupInfoA((LPSTARTUPINFOA)&local_60);
63 GetModuleHandleA((LPCSTR)0x0);
64 local_6c = FUN_00408140();
65 /* WARNING: Subroutine does not return */
66 exit(local_6c);
67 }
68

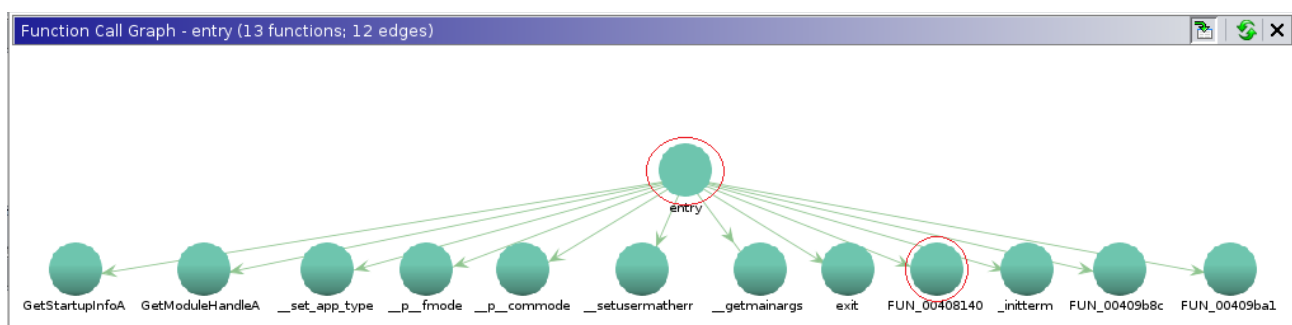
```

Highlight the function and from the Ghidra taskbar, click the Windows tab, scroll down, and select Function Call Graph.

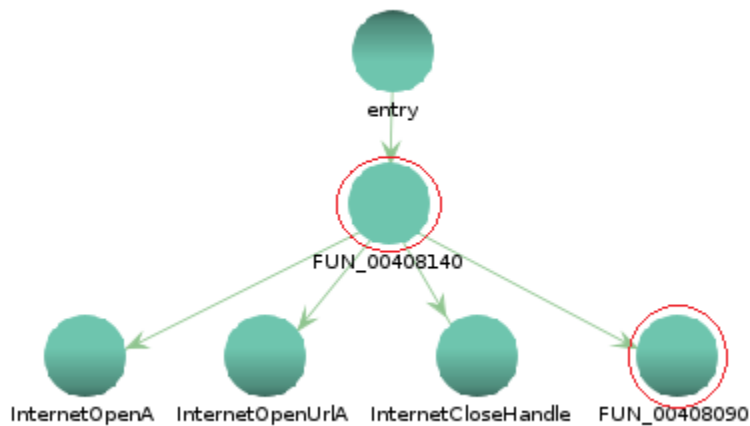


Once we open the Function Graph Call, we can see what the call from the **entry** function looks like.

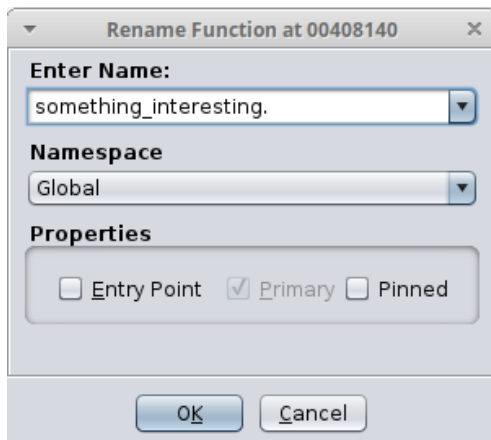
The Function Graph Call tool is a great way to see what functions are being called on by another function. Our function of interest is **FUN_00408140**.



If we x2 click the FUN_00408140 function in the Graph window, we can see what functions it is calling on.



Minimize the Function Graph Call window. Inside the Decompile window, right-click on the FUN_00408140 function and rename it to **something_interesting**. Click OK.



```

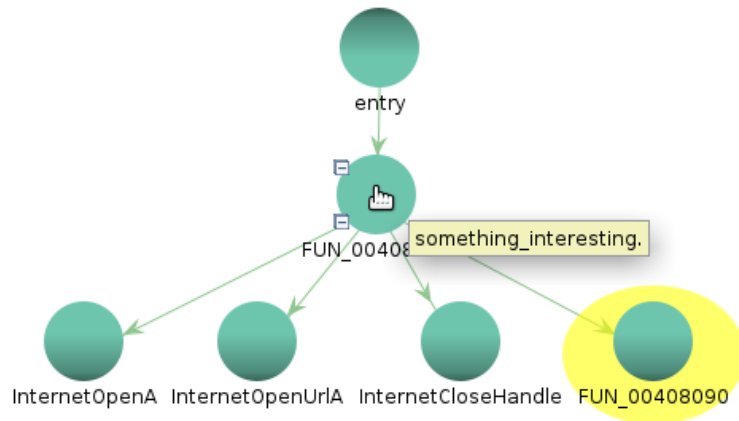
59 LAB_00409b09:
60 } while ((*local_78 != 0) && (*local_78 < 0x21));
61 local_60.dwFlags = 0;
62 GetStartupInfoA((LPSTARTUPINFOA)&local_60);
63 GetModuleHandleA((LPCSTR)0x0);
64 local_6c = something_interesting();
65 /* WARNING: Subroutine does not return */
66 exit(local_6c);
67 }
68

```

X2 Click the function, **something_interesting**. Notice the strange string on line 22 calling on a URL.

<pre> 00408145 MOV ECX,0xe 0040814a MOV ESI,s_http://www.iuqerfsodp9ifjaposdfj_004313d0 0040814f LEA EDI=>local_50,[ESP + 0x8] 00408153 XOR EAX,EAX 00408155 MOVSD ES:EDI,ESI=>s_http://www.iuqerfsodp9ifjaposdfj... 00408157 MOVSB ES:EDI,ESI=>s_http://www.iuqerfsodp9ifjaposdfj... 00408158 MOV dword ptr [ESP + local_17],EAX 0040815c MOV dword ptr [ESP + local_13],EAX 00408160 MOV dword ptr [ESP + local_f],EAX </pre>	<pre> 21 iVar2 = 0xe; 22 puVar3 = (undefined4 *)s_http://www.iuqerfsodp9ifjaposdfj_004313d0; 23 puVar4 = local_50; 24 while (iVar2 != 0) { 25 iVar2 = iVar2 + -1; 26 *puVar4 = *puVar3; 27 puVar3 = puVar3 + 1; 28 puVar4 = puVar4 + 1; 29 } </pre>
---	--

"s_http://www.iuqerfsodp9ifjaposdfj_004313d0"



We can see that the function, **something_interesting** is calling on the two functions, **InternetOpenA()** and **InternetOpenUrlA()**.

InternetOpenA() function is used to initialize the application's use of the WinINet functions.

The **InternetOpenUrlA() function** opens a resource specified by a complete FTP or HTTP URL.

Read the following Microsoft documentation to learn more about these two functions. You will need the code from the pages for the functions [InternetOpenA\(\)](#) and [InternetOpenUrlA\(\)](#).

In the Decompile window, scroll to the bottom and find the functions, InternetOpenA() and InternetOpenUrlA().

```

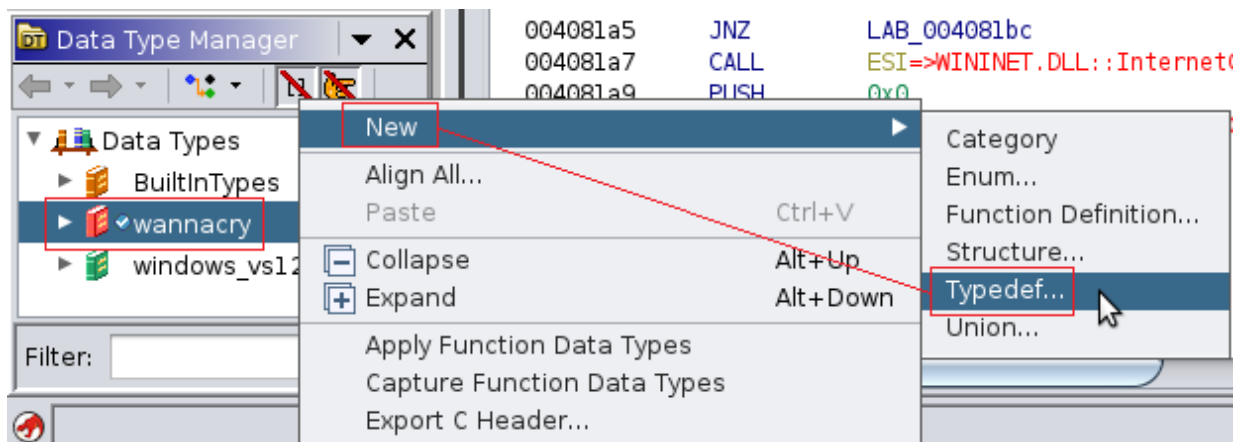
39  uStack100 = 0;
40  local_1 = 0;
41  uVar1 = InternetOpenA(0,1);
42  iVar2 = InternetOpenUrlA(uVar1,&uStack100,0,0,0x84000000,0);
43  if (iVar2 == 0) {
44      InternetCloseHandle(uVar1);
45      InternetCloseHandle(0);
46      FUN_00408090();
47      return 0;
48  }
49  InternetCloseHandle(uVar1);
50  InternetCloseHandle(iVar2);
51  return 0;
52  }
53

```

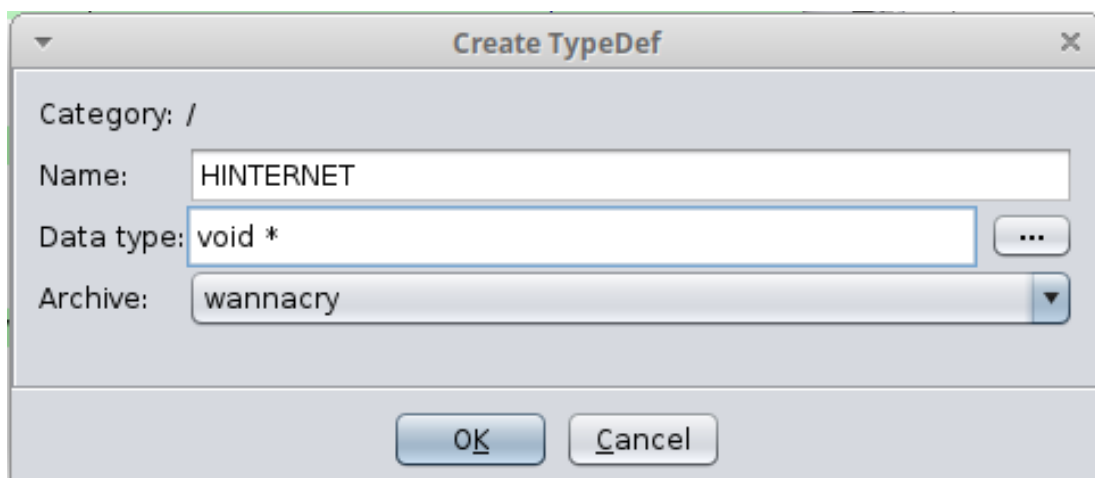
Change the function signature for both functions.

Before we edit the signature, we first need to create a new **typedef** called **HINTERNET**

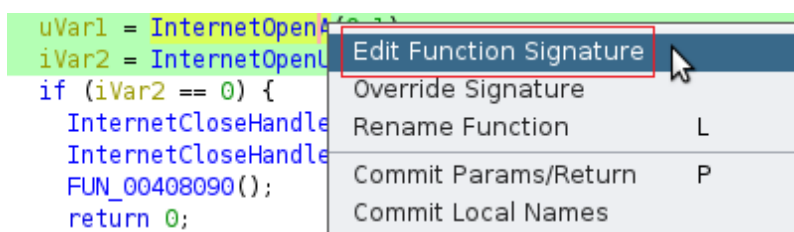
From the Data Type Manager window, right-click on the book labeled, **wannacry** and from the context menu, select New>



Type the following information in the text boxes. Click OK.



Starting with the InternetOpenA() function, we right-click on the function, and from the context menu we select, Edit Function Signature.



From the [Microsoft site](#), we copy and paste the following data into the text box.

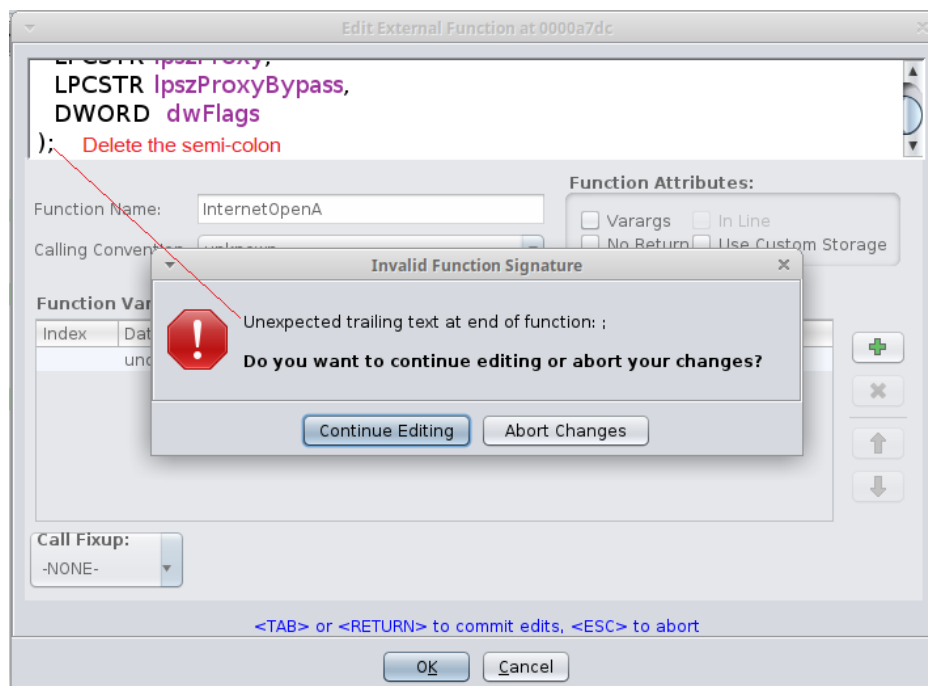
```

C++

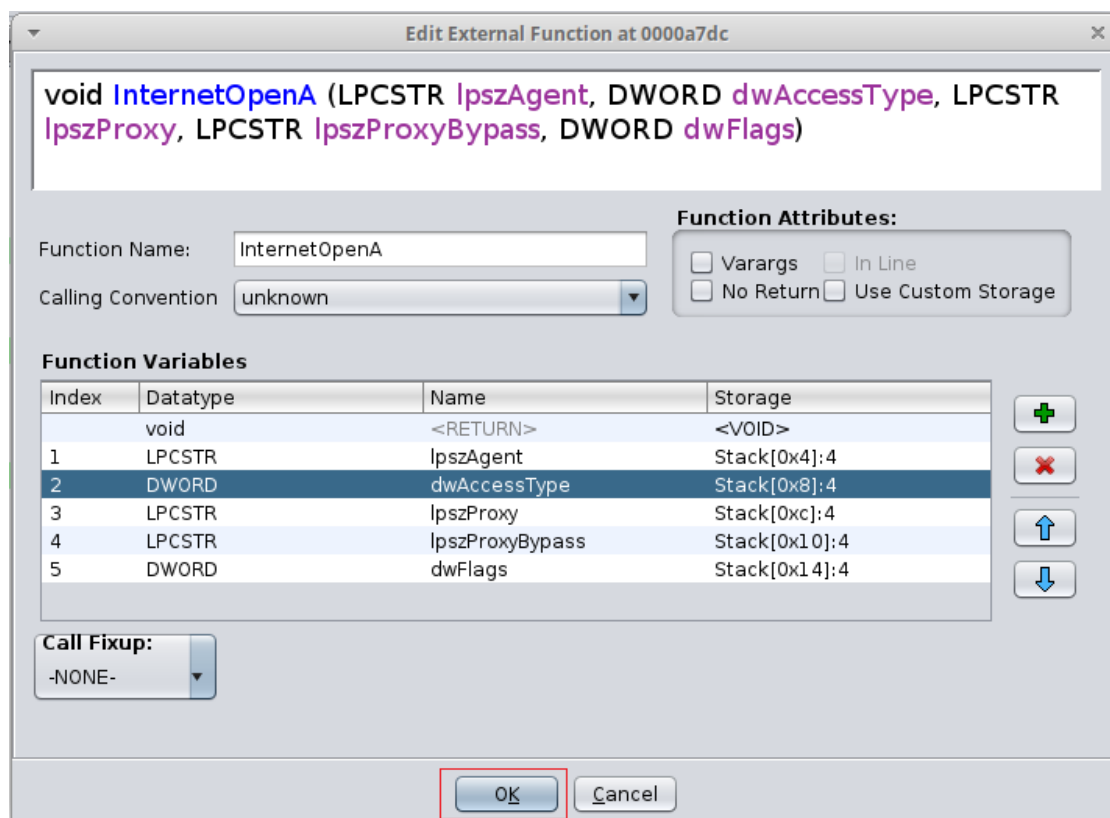
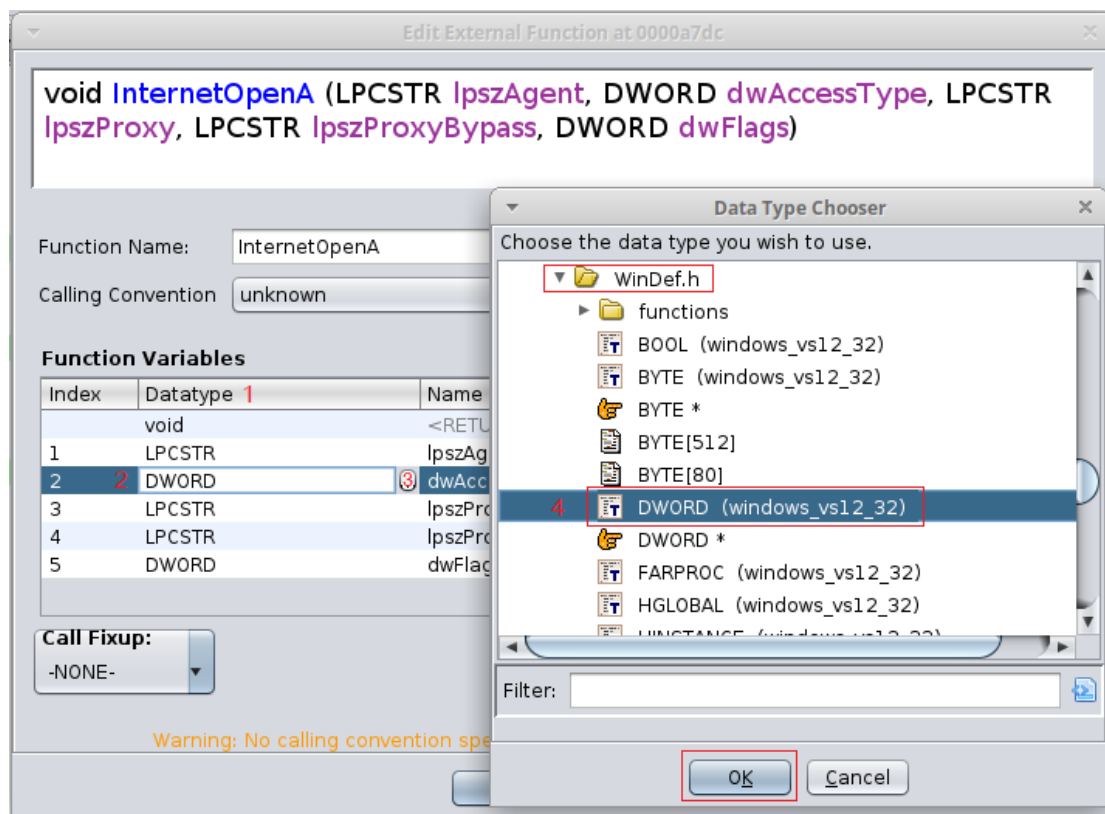
HINTERNET InternetOpenA(
    LPCSTR lpszAgent,
    DWORD dwAccessType,
    LPCSTR lpszProxy,
    LPCSTR lpszProxyBypass,
    DWORD dwFlags
);

```

At the end of the text box is a semi-colon that must be removed.



In the Function Variables, click on Datatype (1). From the options, click on DWORD (2). Over to the right of DWORD, you will see a small grey browse button (3). Click on the button and scroll to the bottom until you come upon the book icon labeled wanncry. Scroll down the list until you come to the folder labeled **Windef.h**, Select the DWORD entitled, **windows_vs12_32** (4). Click OK. Back at the edit window, click OK one more time.

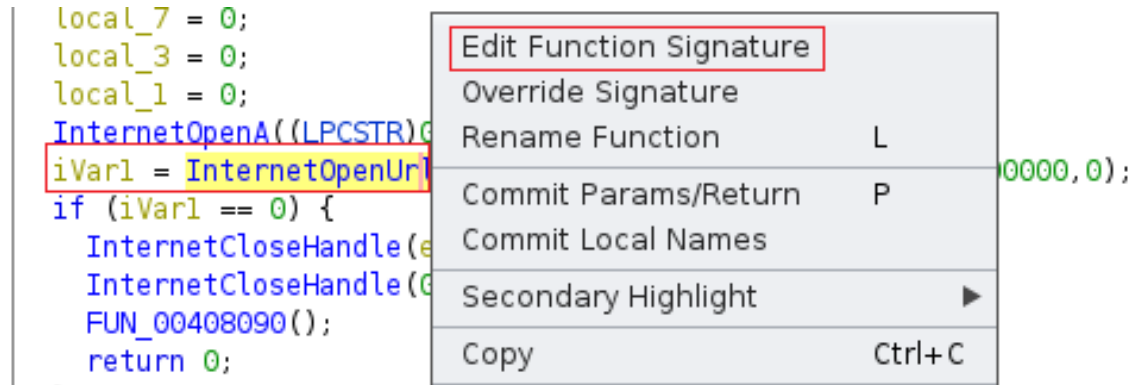


The **InternetOpenA()** has changed to the following.

```
35 | InternetOpenA((LPCSTR)0x0,1,(LPCSTR)0x0,(LPCSTR)0x0,0);
```

We next change the function signature for **InternetOpenUrlA**

We can now change the function signature for InternetOpenUrlA(). From the Decompile window, right-click on the **InternetOpenUrlA()** function, and from the context menu select, Edit Function Signature.



From the [Microsoft site](#), copy and paste the following data into the text box. Remember to Remove the semi-colon at the end of the paste.

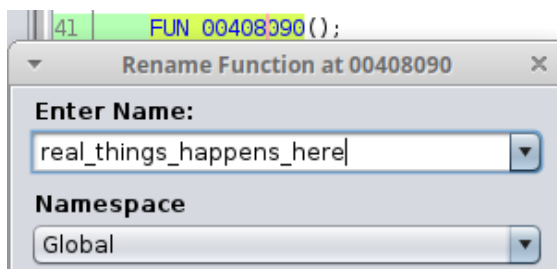
```
C++

HINTERNET InternetOpenUrlA(
    HINTERNET hInternet,
    LPCSTR    lpzUrl,
    LPCSTR    lpzHeaders,
    DWORD     dwHeadersLength,
    DWORD     dwFlags,
    DWORD_PTR dwContext
);
```

If asked to choose a Datatype, accept the default, and click OK. When you are finished, the edited function should look like this.

```
InternetOpenUrlA(hInternet,(LPCSTR)local_50,(LPCSTR)0x0,0,0x84000000,0);
```

Rename the function at line 41 FUN_00408090, to read **real_things_happens_here**.



Looking at line 37 (your line number may differ), we see the function points to the strange URL we discovered earlier.

Line number 38 tells us (your line number may differ) if the request fails to return a null handle, close the handle, and call upon the function, **real_things_happens_here()**.

If the URL request is successful, it closes the handle and quits the program.

Note

Ghidra will not analyze the same file the same way twice. If you close and restart the analysis, the Decompiler windows will show you a different analysis than before. This is Ghidra's best estimation of what the original code would look like.

```
36 | InternetOpenA((LPCSTR)0x0,1,(LPCSTR)0x0,(LPCSTR)0x0,0);
37 | InternetOpenUrlA(hInternet,(LPCSTR)local_50,(LPCSTR)0x0,0,0x84000000,0);
38 | if (hinternet_return == 0) {
39 |     InternetCloseHandle(hInternet);
40 |     InternetCloseHandle(0);
41 |     real_things_happens_here();
42 |     return 0;
43 | }
44 | InternetCloseHandle(hInternet);
45 | InternetCloseHandle(hinternet_return);
46 | return 0;
47 | }
```

Summary –

This is the famous killswitch discovered by Marcus Hutchins, aka MalwareTech. He reversed Engineered the WannaCry and discovered that the ransomware checks for the above URL. However, the URL had never been registered and was inactive. Once the ransomware checked the URL and found it was active, the ransomware was shut down.